Proceedings of 10th IEEE International Conference
on CYBER Technology in Automation, Control, and Intelligent Systems
October 10 - 13, 2020, Xi'an, China

PR-RRT*: Motion Planning of 6-DOF Robotic Arm Based on Improved RRT Algorithm

Yi liang, Hengyang Mu, Diansheng Chen, Xiaodong Wei and Min Wang

***Abstract*—Based on RRT* algorithm, an improved sampling-based motion planning algorithm is proposed in this paper. Firstly, the kinematics analysis of the Schunk lwa3 manipulator is performed, and the D-H model is built. Then the forward kinematics solution of the manipulator is obtained, and the working space of the manipulator is solved based on the Monte Carlo method. Finally, based on the classic RRT motion planning algorithm and the rewiring method in RRT*, the probabilistic-root rapidly-exploring random tree motion planning algorithm is designed and verified by experiments.**

I. INTRODUCTION

Forward kinematics analysis should be performed, and then a detection whether the robotic arm interfered with itself or an obstacle is needed. Finally, the motion planning algorithm should be conducted to get a trajectory. The overall steps for motion planning of a robotic arm are shown in Fig. 1.

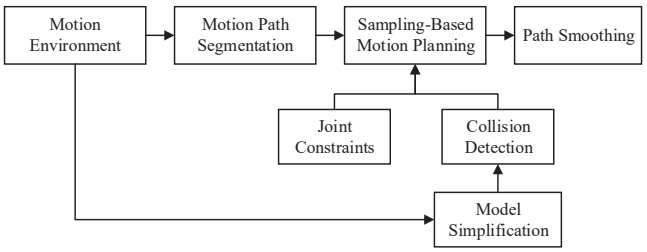

Fig. 1. Robot motion planning flowchart

RRT is a classic sampling-based motion planning algorithm. There are two directions for the improvement of the RRT algorithm: one is improving the existing steps of RRT algorithm, and the other is to add additional steps into the algorithm.

There are many explorations in the first direction, and the improvement can be divided into three types: improvement on sampling method, tree growth method and collision detection method. RRT* algorithm [1] and its improved algorithms [2], [3], [4] reduce useless searches and increase the accuracy of the sampling process by adding a heuristic process in sampling. JT-RRT[5] uses Jacobian transpose matrix in the growth step of random tree, which reduces the planning time. There are also some results of the exploration in the second direction. Optimization steps are added into the RRT, making the algorithm a better motion planning algorithm for some special application [6].

The common advantage of the above RRT-based motion planning algorithm is the non-necessity of considering the singularity problem of the manipulator. In some algorithms, the problem of poor path quality caused by the randomness is also improved to a certain degree. However, the uncertainty of random search and the inadaptability to narrow intersections makes the efficiency of path search still low.

II. KINEMATICS ANALYSIS

The forward kinematics modeling of the manipulator is first performed. Because the joint coordinates of the initial point and the goal point are included in the given conditions of the motion planning algorithm designed in this paper, the process of solving inverse kinematics are not involved.

A. Forward Kinematics Analysis

SCHUNK lwa3 manipulator used in this paper is a 7-DOF robotic arm. However, due to the background of the subject, a 6-DOF robotic arm is required as a research platform. Therefore, according to a common scheme of a six-axis industrial robot, the third axis of the manipulator is fixed, and the remaining six rotation joints are retained.

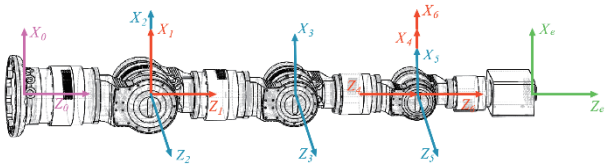

Fig. 2. D-H coordinate system on manipulator

The D-H coordinate system is established as shown in Fig. 2. Then the D-H parameters of each joint according to the coordinate system are calculated. D-H parameter measurement is conducted on the SCHUNK manipulator as shown in

TABLE I.

Homogeneous transformation matrix of the adjacent links from $\{O_i - x_i y_i z_i\}$ to $\{O_{i-1} - x_{i-1} y_{i-1} z_{i-1}\}$ is:

$$A_i = \begin{bmatrix} \cos\theta_i & -\sin\theta_i\cos\alpha_i & \sin\theta_i\sin\alpha_i & a_i\cos\theta_i \\ \sin\theta_i & \cos\theta_i\cos\alpha_i & -\cos\theta_i\sin\alpha_i & a_i\sin\theta_i \\ 0 & \sin\alpha_i & \cos\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (1)$$

Research supported by the National Key R&D Program of China (2018YFB1309303).

978-1-7281-9010-5/20/$31.00 © 2020 IEEE

TABLE I. D-H Parameters of the SCHUNK Manipulator

Link	$\alpha(°)$	$a(mm)$	$d(mm)$	$\theta(°)$	Range of θ (°)	Range of Joint (°)
1	90	0	307.0	$\theta_1(0)$	[-180,180]	[-180,180]
2	0	328.0	0	$\theta_2(90)$	[-30,210]	[-120,120]
3	90	0	0	$\theta_3(90)$	[-30,210]	[-120,120]
4	90	0	276.5	$\theta_4(0)$	[-180,180]	[-180,180]
5	-90	0	0	$\theta_5(0)$	[-120,120]	[-120,120]
6	0	0	345.0	$\theta_6(0)$	[-180,180]	[-180,180]

The D-H parameter is put into (1) to obtain the conversion matrix of joint 1 to joint 6. By multiplying these transformation matrices from left to right, the transformation matrix of the complete manipulator can be obtained:

$$ {}^{1}_{6}T = A_1A_2A_3A_4A_5A_6 = {}^{0}_{6}T \tag{2} $$

Where ${}^{0}_{6}T$ is the transformation matrix of coordinate system {6} relative to coordinate system {0}. According to a common method [7], it is expressed as:

$$ {}^{0}_{6}T = \begin{bmatrix} \boldsymbol{n} & \boldsymbol{o} & \boldsymbol{a} & \boldsymbol{p} \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} n_x & o_x & a_x & p_x \\ n_y & o_y & a_y & p_y \\ n_z & o_z & a_z & p_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{3} $$

By combining (2) and (3), the final positive kinematics solution formula can be obtained as:

$$ \begin{cases} n_x = (s_1c_4 - c_1c_{23}s_4)s_6 + \left((s_1s_4 + c_1c_{23}c_4)c_5 + c_1s_{23}s_5\right)c_6 \\ n_y = -(c_1c_4 + s_1c_{23}s_4)s_6 - \left((c_1s_4 - s_1c_{23}c_4)c_5 - s_1s_{23}s_5\right)c_6 \\ n_z = -(c_{23}s_5 - s_{23}c_4c_5)c_6 - s_{23}s_4s_6 \end{cases} \tag{4} $$

$$ \begin{cases} o_x = (s_1c_4 - c_1c_{23}s_4)c_6 - \left((s_1s_4 + c_1c_{23}c_4)c_5 + c_1s_{23}s_5\right)s_6 \\ o_y = \left((c_1s_4 - s_1c_{23}c_4)c_5 - s_1s_{23}s_5\right)s_6 - (c_1c_4 + s_1c_{23}s_4)c_6 \\ o_z = (c_{23}s_5 - s_{23}c_4c_5)s_6 - s_{23}s_4c_6 \end{cases} \tag{5} $$

$$ \begin{cases} a_x = c_1s_{23}c_5 - (s_1s_4 + c_1c_{23}c_4)s_5 \\ a_y = (c_1s_4 - s_1c_{23}c_4)s_5 + s_1s_{23}c_5 \\ a_z = -c_{23}c_5 - s_{23}c_4s_5 \end{cases} \tag{6} $$

$$ \begin{cases} p_x = d_4c_1s_{23} - d_6\left((s_1s_4 + c_1c_{23}c_4)s_5 - c_1s_{23}c_5\right) + a_2c_1c_2 \\ p_y = d_4s_1s_{23} + d_6\left((c_1s_4 - s_1c_{23}c_4)s_5 + s_1s_{23}c_5\right) + a_2s_1c_2 \\ p_z = a_2s_2 + d_1 - d_4c_{23} - d_6(c_{23}c_5 + s_{23}c_4s_5) \end{cases} \tag{7} $$

Where c_i and s_i are the shorthands of $cos\theta_i$ and $sin\theta_i$ respectively, while c_{ij} and s_{ij} are the shorthands of $cos(\theta_i+\theta_j)$ and $sin(\theta_i+\theta_j)$ respectively.

B. Workspace Solution

Monte Carlo method is used to solve the workspace. All joint variables are randomly selected within the range, and then the end position can be obtained. 100000 random joint points are generated in the MATLAB simulation environment, and its distribution can be observed as shown in Fig. 3.

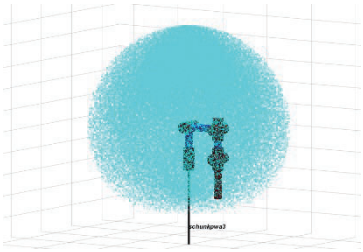

Fig. 3. Working space point cloud of the end

The range of the points of the envelope of the point cloud on each coordinate axis is [-941.7mm, 941.7mm] on x axis and y axis, and [-466.3mm, 1241.7mm] on z axis.

III. Collision Detection Method

There are many studies on collision detection algorithms. For different robot types, working scenarios and real-time requirements, the collision detection methods are different [8], [9], [10]. The bounding box method simplifying the robot and environment objects is commonly used to reduce the excessive calculation [11]. The collision detection of simplified models is simple and fixed, and the efficiency is greatly improved.

A. Collision Detection Based on Cylindrical Envelope

The closest simple geometry that replaces the links of a robotic arm is a cylinder. Therefore, a cylindrical envelope can be used for collision detection for links. The collision problem between the links of the robotic arm is converted into a problem of judging whether a collision occurs between the cylinders. The problem is simplified.

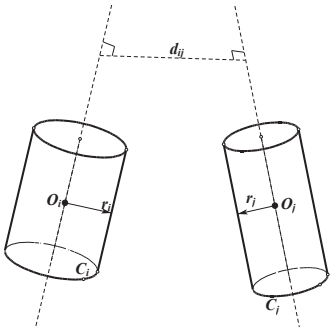

Fig. 4. Position relationship between two cylinders

A key step in using the cylindrical envelope method is to find the size, position, and attitude of the envelope cylinder that replaces the real link or other items, as shown in Fig. 4. The position of the centerline of the cylinder in three-dimensional space is solved below. The center line of the outer cylinder of the manipulator connecting rod is set to C_i, and the space straight line equation where C_i is located will be expressed as:

$$ \frac{x-x_i}{x_{i+1}-x_i} = \frac{y-y_i}{y_{i+1}-y_i} = \frac{z-z_i}{z_{i+1}-z_i} = t,\ t \in (0, 1) \tag{8} $$

Where the (x_i, y_i, z_i) and $(x_{i+1}, y_{i+1}, z_{i+1})$ are the end

face center coordinates of the link envelope cylinder, and t is a proportionality constant.

Converting (8) into a parametric form, we get:

$$\begin{cases} X_i(t)=tx_{i+1}+(1-t)x_i \\ Y_i(t)=ty_{i+1}+(1-t)y_i \\ Z_i(t)=tz_{i+1}+(1-t)z_i \end{cases} \quad ,t\in(0,1) \tag{9}$$

By simplification, the collision between two links becomes the distance between the centerlines of the two cylinders and the relationship between the radius of the two cylinders. According to (9), and it can be known that the spatial vertical distance between the centerlines of two cylinders is:

$$d_{ij}=\sqrt{\left(X_i(t)-X_j(m)\right)^2+\left(Y_i(t)-Y_j(m)\right)^2+\left(Z_i(t)-Z_j(m)\right)^2} \tag{10}$$

When $d_{ij}>r_i+r_j$, the two cylinders will not collide. When $d_{ij}<r_i+r_j$, the two cylinders will collide.

The first link is fixed and each other link are represented as a cylinder, as shown in Fig. 5. When determining the size of the enveloping cylinder of each link, it is necessary to ensure that the connecting rod is completely enveloped while ensuring that its size is as small as possible, so as to ensure the true validity of the collision detection method of the simplified model. At the same time, the working space loss of the robotic arm is minimized as much as possible. Therefore, the radius of the cylinder adopts the minimum full envelope radius of the link, and the height of the cylinder is the length of the link, and both increase a certain safety factor k to be 1.05.

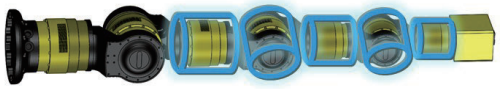

Fig. 5. Envelope cylinders of links

The specific dimensions of the envelope cylinder are shown in the TABLE II.

TABLE II. Dimensions of the Envelope Cylinders

Represented Link	Radius (mm)	Axial Length (mm)
2	69	194
3	55	180
4	55	171
5	53	158
6	53	148

B. Design of Collision Detection Method

Obstacles in the robot's working environment are mostly structured objects, and their positions are relatively fixed. And most of the shapes can be approximated by simple rectangular parallelepipeds. Objects are arranged regularly and are mostly distributed in rows and columns.

The Axis-Aligned Bounding Boxes (AABB) method is a method that enclosing objects with a cuboid. Each side of the cuboid is parallel to the coordinate axis of the world coordinate system, and the length of each side is the minimum side length that can envelop the object. Therefore, a three-dimensional coordinate system is established therein, and the AABB bounding box is most in line with the shape of the actual obstacle, and the calculation of collision detection is reduced.

Based on the above analysis, the collision detection method combining the cylindrical envelope method with the AABB method is adopted. Specifically, the cylindrical envelope is used for each link of the robot arm, and the AABB is used for obstacles. The collision detection process of the sorting robot in a certain state is shown in Fig. 6: the information of the obstacle enclosure is stored in a preset file, which is read when the collision detection algorithm runs, and at the same time, each link of the robot arm The position of the cylindrical envelope is solved, that is, the centerline equation is solved; after each object model is obtained, collision detection is performed, including the detection of the collision of the cylinder between the links of the robotic arm and the collision of the cylinder and the cuboid between the robotic arm and the obstacle.

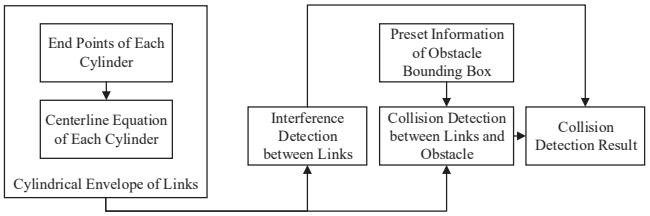

Fig. 6. Flowchart of collision detection

If any one of the detected results is a collision, the final result of the collision detection algorithm is a collision; if no collision is detected inside the robotic arm, between the robotic arm and the obstacle, the final result is no collision.

IV. Motion Planning Algorithm

Common end pose spatial planning algorithms include artificial potential field method, ant colony algorithm [12], etc. The advantage of planning in the end pose space is that the end path is intuitive and clear. The disadvantage is that the inverse kinematics needs to be solved constantly, and the complexity of the calculation process increases exponentially with the increase of the robot's degree of freedom. In order to conveniently add joint constraints and collision detection to the planning and speed up the planning speed, motion planning is selected in joint space. Existing sampling-based motion planning algorithms include stochastic road map method, fast expanding random tree method, SBL [13], [14], etc.

A. RRT Algorithm and its Improvement

The basic step of RRT is to grow a tree from the starting point of the path, and continuously expand the leaf nodes to find the target point by growing randomly. Finally, when a growing leaf node reaches or near the target point, a path is successfully obtained. The nodes of the point continuously seek the path of the parent node in reverse to get the final planning result. The main improvement of the RRT-connect algorithm is that an extended tree is also introduced at the target point, and the tree of the starting point and the target point is randomly expanded at the same time in each search iteration. The method is complete and not optimal.

Due to the large randomness of the path in the RRT algorithm, the planning result is often much more expensive than the optimal path. Therefore, improved RRT motion planning algorithm is considered this paper. By analyzing the RRT algorithm and researching and summing up the improved algorithm, consider improving it from three aspects: (a) the way of sampling; (b) the selection of the nearest point; (c) the way of expanding the tree.

B. Improved RRT Algorithm——PR-RRT* Algorithm

According to theoretical analysis, in the case of completely random exploration, the relationship between the size of the area to be explored and its distance from the goal point is:

$$\lim_{n\to\infty} A_{probe} = \pi l_{dis}^2 \tag{14}$$

Where n is the number of the explorations, l_{dis} is the distance between the initial point and the goal point, and A_{probe} is the area of the exploration points.

It is concluded that as the exploration distance increases, the number of explorations increases in a square order. It can be seen that the method of increasing the number of trees can reduce the exploration distance of a single tree and thus greatly reduce the number of explorations.

Therefore, the idea of improving the algorithm in this paper is mainly to increase the root of the tree. The Probabilistic Root Rapidly-exploring Random Tree (PR-RRT*) motion planning algorithm is proposed.

The collision detection method of a point and a straight path is firstly designed. The collision detection method for a point is the collision detection algorithm designed in this paper. As for the straight path, five points are sampled on the entire path by average sampling, and collision detection is performed for each point. If there is no collision in all five points, it is inferred that the path has no collision.

As shown in Fig. 7, a certain number of points are randomly generated in the workspace at the beginning, and they are the candidate points for the tree roots. Then the points are connected and collision detection are performed on each edge, and collision-free edges remain. After that, the sum of projection lengths of all the edges are calculated, and the points with the largest sum are selected as the roots, as the yellow dots shown in the Fig. 7. The reason of using this strategy to select the root is that the points suitable as roots are distributed at key locations on the map, such as obstacle corners, narrow intersections, etc.

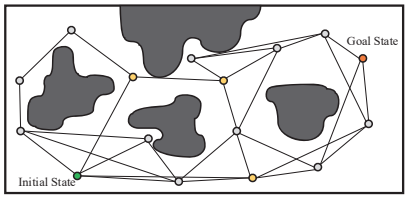

Fig. 7. Generation and selection of random roots

The roots of the tree include the initial point and the goal point in addition, as the green and red points in Fig. 7.

After all the random roots are selected, they start to grow simultaneously, as shown in Fig. 8. The new point is generated by randomly sampling each tree as it grows, and its connection target is the other tree closest to it. Heuristic strategies are used to generate new points, and probabilistic greedy strategies are used to reduce invalid exploration.

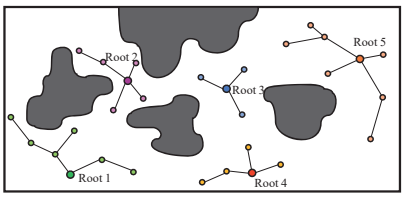

Fig. 8. Multiple trees growing simultaneously

The growth method of each tree of PR-RRT is also different from the ordinary RRT algorithm, but same as the method used in in RRT* algorithm. It is called rewiring method, which adds an optimization to the parent node selection of each new point, reducing the path distance of each generation.

If the path exists, the start tree will finally be connected with the goal tree, as shown in Fig. 9.

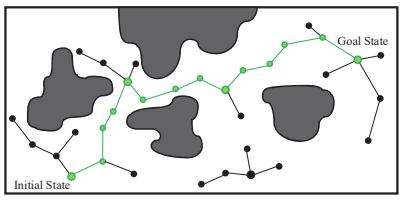

Fig. 9. Path generating

The resulting path is then smoothed to eliminate redundant path points. The results are shown in Fig. 10. The specific processing adopts greedy algorithm.

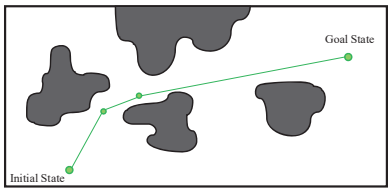

Fig. 10. Path smoothing

The flowchart of the PR-RRT algorithm proposed in this paper is shown in Fig. 11.

V. Experiment Results

A. Simulation Experiment

In order to facilitate the simulation in the Windows 10 operating system, a simulation environment of the sorting environment is built based on V-rep. By modifying the motion planning source code of the RRT* included in the OMPL library, the PR-RRT motion planning algorithm is implemented.

A simple warehouse sorting environment model is built in a virtual environment, as shown in Fig. 12. It contains a check box, two logistics order boxes and simplified models for environmental obstacles such as roller lines and camera.

In the simulated logistics warehouse sorting environment, a motion planning algorithm is used on a simulated six-axis robotic arm. The sorting path in the simulation environment is simulated, where the starting point is the state where the end-effector reaches the check box, and the target point is the state where the end-effector reaches the putting point above the order box. Using the PRM and RRT* algorithms as references, the feasibility and actual performance of the PR-RRT motion planning algorithm in planning this path is tested. The result is shown in TABLE III.

TABLE III. Result of Simulation Experiment

Times	Planning Time (s)			Motion Time (s)		
	PRM	RRT*	PR-RRT	PRM	RRT*	PR-RRT
1	5.01	0.81	1.51	12.05	11.64	8.13
2	4.96	0.98	1.53	11.88	10.21	9.02
3	5.11	0.97	1.55	13.51	13.21	8.53
4	Failed	1.05	1.49	Failed	13.11	8.66
5	3.55	0.93	1.80	10.64	11.87	9.52
6	3.58	0.96	1.53	13.28	11.31	8.09
7	5.25	1.06	1.51	12.87	11.73	8.76
8	5.19	0.90	1.43	13.33	12.11	9.29
9	5.10	0.83	1.36	10.52	11.24	7.95
10	Failed	1.04	1.51	Failed	10.95	9.41
Average	4.72	0.95	1.52	12.26	11.74	8.74

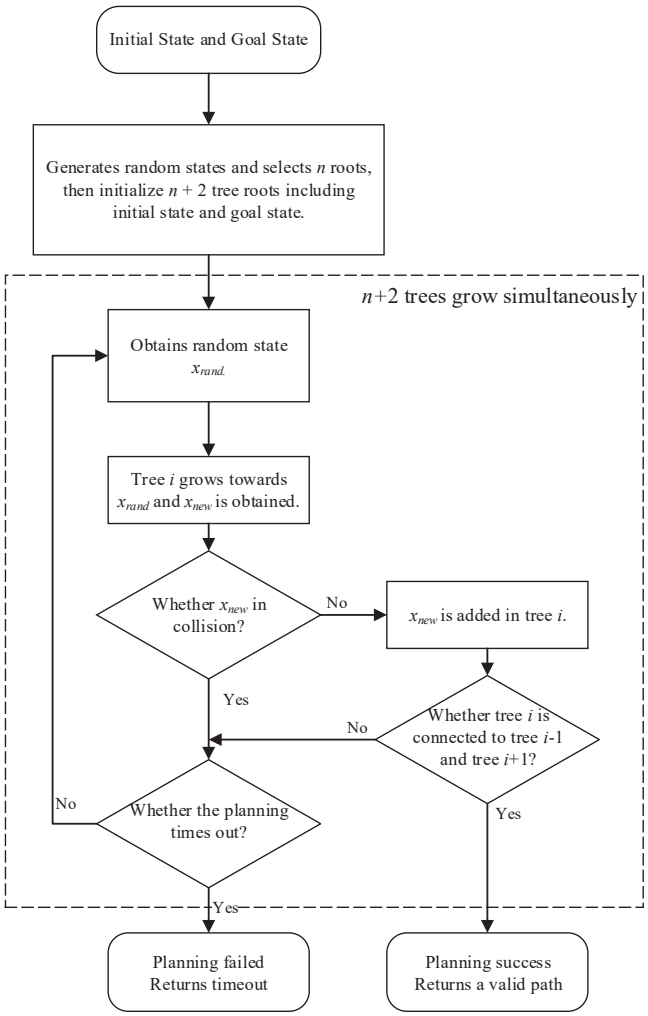

Fig. 11. Flowchart of PR-RRT algorithm

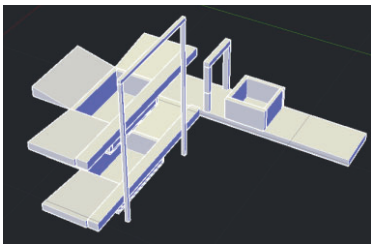

Fig. 12. Simulation sorting environment

It can be known from experiments that the PR-RRT algorithm has a higher success rate, the planning algorithm runs faster, and takes slightly longer than RRT* on average, but the path of PR-RRT is better than those of PRM and RRT*, which verifies the effectiveness of the proposed PR-RRT motion planning algorithm.

B. Real Manipulator Experiment

The motion planning algorithm is implemented in MATLAB. The network connection module is also

developed in the planning interface to connect the manipulator server. The specific experimental environment is shown in Fig. 13, where some boards are used as the obstacle. After successful planning, a robot arm trajectory containing 50 joint coordinate points is sent to the robot arm. After receiving the trajectory, the manipulator starts to pass through 50 coordinate points in sequence continuously, and finally reaches the goal point.

Fig. 13. Real manipulator planning experiment

The experiment carried out real manipulator motion planning experiments. In order to ensure the safety of the experiment, the upper limit of the manipulator movement speed ratio was set to 0.01, and the real operation results of the manipulator are shown in TABLE IV.

TABLE IV. Result of Real Manipulator Experiment

Times	Transmission Time (s)	Motion Time (s)
1	0.56	28.78
2	0.48	28.14
3	0.43	29.12
4	0.60	27.28
5	0.81	29.72
6	0.62	28.60
7	0.51	28.76
8	0.55	28.66
Average	0.57	28.80

It can be seen that the PR-RRT algorithm of the manipulator is applicable to the real manipulator, and the obstacle avoidance performance is stable.

VI. Conclusion

In this paper, the kinematics analysis of the manipulator in the laboratory is carried out, the D-H parameter model is established, and the working space of the manipulator is solved based on the Monte Carlo method. Then the collision detection method combining the cylindrical bounding box with the AABB method is designed. Finally, based on the classical RRT motion planning algorithm and the rewiring method in RRT*, the PR-RRT algorithm is designed. The experiment results show that the algorithm is better than the classical PRM and RRT* algorithm in path cost.

Acknowledgment

This research was supported by the National Key R&D Program of China 2018YFB1309303. The authors would like to thank all the team members.

References

[1] Karaman, Sertac , et al. "Anytime Motion Planning using the RRT*." IEEE International Conference on Robotics and Automation, ICRA 2011, Shanghai, China, 9-13 May 2011 IEEE, 2011.

[2] Gammell, Jonathan D. , S. S. Srinivasa , and T. D. Barfoot . "Informed RRT*: Optimal sampling-based path planning focused via direct sampling of an admissible ellipsoidal heuristic." IEEE/RSJ International Conference on Intelligent Robots & Systems IEEE, 2014.

[3] S. Choudhury, J. D. Gammell, T. D. Barfoot, S. S. Srinivasa and S. Scherer, "Regionally accelerated batch informed trees (RABIT*): A framework to integrate local information into optimal path planning," 2016 IEEE International Conference on Robotics and Automation (ICRA), Stockholm, 2016, pp. 4207-4214.

[4] Kim, Min Cheol , and J. B. Song . "Informed RRT* with improved converging rate by adopting wrapping procedure." Intelligent Service Robotics 11.1(2018):53-60.

[5] Vandeweghe, J Michael , D. Ferguson , and S. Srinivasa . "Randomized Path Planning for Redundant Manipulators without Inverse Kinematics." IEEE-RAS International Conference on Humanoid Robots IEEE, 2007.

[6] Kuffner, J. J. , and S. M. Lavalle . "RRT-connect: An efficient approach to single-query path planning." Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation. Symposia Proceedings (Cat. No.00CH37065) IEEE, 2002.

[7] Z. G. Wang. "Research on obstacle avoidance path planning for 6-DOF manipulator." Southwest Jiaotong University, 2018.

[8] D. W. Ma, W. Ye, and Y. Li. "Survey of box-based algorithms for collision detection." Journal of System Simulation, 04(2006):246-249+252.

[9] Y. H. Dai. "Research on obstacle avoidance path planning for 6-DOF manipulator." Southwest University of Science and Technology, 2013.

[10] Y. S. Zou, et al. "Survey on real-time collision detection algorithms." Application Research of Computers, 25.1(2008):8-12.

[11] L. M. Bian, et al. " Survey of bounding box collision detection technology." Mechanical Management and Development, 23.2(2008):27-28.

[12] D. Q. Zhu, and M. Z. Yan. " Survey on technology of mobile robot path planning." Control and Decision, 07(2010):4-10.

[13] Gildardo Sánchez-Ante, and Jean-Claude Latombe. "A single-query bi-directional probabilistic roadmap planner with lazy collision checking." International Symposium Robotics Research 2001.

[14] Ioan A. Şucan, and L. E. Kavraki . "A sampling-based tree planner for systems with complex dynamics." IEEE Transactions on Robotics 28.1(2012):116-131.

[15] Sergey Brin. "Near neighbor search in large metric spaces." VLDB, 1995.